

MAXIMUS

Module dédié et synchronisation centralisée

Fonctionnement détaillé de l'enrôlement, du bootstrap, de la synchronisation des modules et des données d'entreprise entre MAXIMUS principal et une installation Windows locale.

Protocole de synchronisation v2

Laravel / PostgreSQL

Windows / PHP cURL

Document technique

Public cible : ingénieur logiciel, ingénieur système, administrateur de déploiement.

Périmètre : installation `dedicated` ou `on_premise`, pilotée par MAXIMUS principal en mode `central`.

Date de référence : 21 septembre 2026.

Ce document décrit le comportement du code présent dans le dépôt au moment de sa génération. Les chemins de fichiers sont relatifs à la racine du dépôt.

Résumé exécutif

MAXIMUS fonctionne selon deux périmètres complémentaires :

- **MAXIMUS principal** : installation centrale multi-entreprises. Elle conserve les entreprises, les accès modules, le catalogue publié et les enregistrements d'installations.
- **Installation dédiée** : instance locale ou on-premise limitée à une seule entreprise. Elle reçoit une copie contrôlée de la configuration de cette entreprise et continue à fonctionner avec sa dernière configuration valide si le central est momentanément indisponible.

La synchronisation n'est pas une réplication SQL et ne donne jamais accès à la base centrale. L'installation dédiée appelle un endpoint HTTPS central avec un jeton d'installation. Le central vérifie le jeton par son hash, identifie l'installation, reconstruit une **photo complète** de la configuration autorisée, puis la renvoie. L'installation locale vérifie l'identité, le protocole, la version et le catalogue avant d'appliquer la photo dans une transaction PostgreSQL.

Principe important : l'accès effectif à un module est reconstruit depuis les lignes persistées de `maximus_company_modules`. La demande initiale d'inscription n'est qu'un repli historique ; elle ne suffit pas à décrire une activation réalisée plus tard par MAXIMUS.

Table des matières

1. Vocabulaire et modes d'installation
2. Architecture générale et frontières de confiance
3. Cycle de vie complet : émission, bootstrap, synchronisation
4. Enrôlement central et génération du bootstrap
5. Installation locale et validation initiale stricte
6. Contrat HTTP de synchronisation
7. Traitement local détaillé et transaction d'application
8. Persistance locale, fonctionnement hors ligne et heartbeat
9. Scheduler Windows et comportement opérationnel
10. Sécurité et modèle de menace
11. États, erreurs et diagnostic
12. Versionnement et compatibilité
13. Procédures d'exploitation
14. Cartographie du code et invariants à préserver

1. Vocabulaire et modes d'installation

Terme	Signification
Central / principal	Instance en mode <code>MAXIMUS_DEPLOYMENT_MODE=central</code> . Elle expose l'administration MAXIMUS, les entreprises, le catalogue et les endpoints d'installation.
Installation dédiée	Instance en mode <code>dedicated</code> . Elle sert une seule entreprise et son accès utilisateur est borné par <code>MAXIMUS_INSTALLATION_COMPANY_ID</code> .
Installation on-premise	Même modèle d'isolation qu'une dédiée, avec un mode <code>on_premise</code> adapté à un hébergement local ou contrôlé par l'entreprise.
Entreprise	Tenant métier identifié par <code>companyId</code> . Une installation ne peut synchroniser que cette entreprise.
Installation	Instance technique identifiée par <code>installationId</code> . Une entreprise peut avoir plusieurs installations depuis la migration autorisant plusieurs lignes.
Bootstrap	Manifeste d'enrôlement transmis une seule fois à l'installateur. Il contient l'URL centrale, l'identité, les versions et le jeton en clair.
Photo de configuration	Payload complet renvoyé par <code>GET /api/installation-sync/configuration</code> . Il décrit l'entreprise, ses modules, le catalogue local, ses domaines et ses accès ERP.

2. Architecture générale et frontières de confiance



La frontière est volontairement orientée : le local **tire** la configuration ; le central ne pousse pas de connexion entrante vers le PC Windows. Le serveur central ne reçoit donc pas besoin d'un port entrant sur le réseau de l'entreprise. En retour, le local envoie uniquement un heartbeat après application de la configuration.

2.1 Responsabilités du central

- autoriser un administrateur MAXIMUS à créer ou renouveler une installation ;
- stocker uniquement le hash SHA-256 du jeton ;
- conserver l'association installation → entreprise et le mode d'installation ;
- reconstruire les modules actifs depuis les autorisations persistées ;
- publier uniquement le catalogue validé ;
- révoquer une installation et refuser les appels ultérieurs ;
- observer les dernières connexions et versions de configuration.

2.2 Responsabilités du local

- conserver le jeton et les identifiants de l'installation hors de l'interface utilisateur ;
- appeler le central périodiquement ;
- refuser tout payload qui ne correspond pas à son entreprise ou son installation ;
- valider le protocole, la version de build et le catalogue avant écriture ;
- appliquer la configuration de façon transactionnelle ;
- continuer avec la dernière configuration valide en cas d'échec réseau.

3. Cycle de vie complet

```
1. ADMIN MAXIMUS
   | crée / renouvelle une installation
   ▼
2. CENTRAL : ligne maximus_installations + token_hash
   | renvoie le bootstrap une seule fois
   ▼
3. TRANSFERT HORS BANDE DU BOOTSTRAP
   | fichier JSON protégé vers le serveur Windows
   ▼
4. INSTALLATEUR LOCAL
   | écrit .env, vérifie le commit et lance l'application initiale
   ▼
5. SYNCHRONISATION INITIALE STRICTE
   | identité + protocole + applicationVersion + packageVersion
   ▼
6. INSTALLATION ACTIVE
   | scheduler toutes les 5 minutes
   ▼
7. SYNCHRONISATION PÉRIODIQUE
   | fetch → validation → transaction → état local → heartbeat
   ▼
8. HORS LIGNE OU RÉVOCATION
   |─ réseau indisponible : dernière configuration conservée
   |─ token révoqué : appels refusés, exploitation à arrêter
   |─ payload invalide : transaction annulée, données précédentes conservées
```

4. Enrôlement central et génération du bootstrap

4.1 Autorisation

Les routes d'installation sont protégées par `maximus.central` et `maximus.auth`. Le contrôleur vérifie en plus que l'acteur authentifié possède le rôle `maximus_admin`. L'entreprise doit être active et non supprimée.

4.2 Routes d'administration

Méthode	Route	Fonction
GET	/api/companies/{companyId}/installation-manifest	Consulter le manifeste disponible côté administration.
POST	/api/companies/{companyId}/installation	Créer ou renouveler l'installation par défaut.
POST	/api/companies/{companyId}/installations/{installationId}	Renouveler une installation précise ou la réémettre.
DELETE	/api/companies/{companyId}/installation[s]/{installationId?}	Révoquer une installation ou toutes les installations ciblées selon la route.

4.3 Création du jeton

À chaque émission, le central génère un nouveau jeton aléatoire préfixé `mxinstall_`. La valeur en clair n'est jamais enregistrée dans la base centrale : seule `hash('sha256', $token)` est persistée dans `maximus_installations.token_hash`.

Une réémission sur une installation existante remplace donc le hash précédent et invalide immédiatement l'ancien jeton. La version de configuration est incrémentée et l'état d'installation revient à `READY`.

4.4 Bootstrap retourné

```
{
  "centralUrl": "https://maximus-erp.onrender.com",
  "installationId": "installation-<uuid>",
  "companyId": "<company-id>",
  "mode": "dedicated",
  "applicationVersion": "<commit-deploy>",
  "syncProtocolVersion": 2,
  "token": "mxinstall_<valeur-secrete>"
}
```

Le bootstrap est un secret d'enrôlement. Il doit être transmis par un canal protégé, ne pas être ajouté à Git, ne pas être copié dans un ticket public et ne pas être affiché dans les logs. Le script d'installation le recopie dans `.env` avec des permissions restreintes.

5. Installation locale et validation initiale

`scripts/install-maximus-instance.sh` installe une archive qui doit correspondre au commit fourni par le bootstrap. Le script collecte le manifeste, vérifie les champs obligatoires et écrit notamment :

Variable	Rôle
MAXIMUS_DEPLOYMENT_MODE	dedicated ou on_premise . Toute autre valeur est refusée pour une installation isolée.
MAXIMUS_CENTRAL_URL	Base HTTPS du central.
MAXIMUS_INSTALLATION_TOKEN	Secret local utilisé dans l'en-tête Bearer.
MAXIMUS_INSTALLATION_ID	Identité technique de cette instance.
MAXIMUS_INSTALLATION_COMPANY_ID	Tenant unique autorisé localement.
MAXIMUS_EXPECTED_APPLICATION_VERSION	Version exigée par le bootstrap initial.
MAXIMUS_BUILD_VERSION	Version de l'archive locale, matérialisée dans MAXIMUS_BUILD_VERSION .

5.1 Preuve de version initiale

Lors de la première installation, `InstallationSyncService::validatePayload()` est appelé avec `$initial=true` . Le local refuse si :

- le mode n'est pas dédié ou on-premise ;
- l'ID entreprise ou l'ID installation du payload ne correspond pas au fichier `.env` ;
- le mode reçu ne correspond pas au mode local ;
- le protocole reçu n'est pas la version `2` ;
- le central ne publie pas une `applicationVersion` ;
- la version attendue ou la version du package local ne correspond pas à celle du central.

Cette preuve empêche d'installer un package qui n'est pas celui pour lequel le bootstrap a été émis. Après une première synchronisation réussie, une différence de commit peut être tolérée si le protocole reste compatible ; elle produit alors un avertissement de version et aucune mise à jour automatique du logiciel.

6. Contrat HTTP de synchronisation

6.1 Authentification

Le local envoie :

```
Authorization: Bearer mxinstall_<secret>
Accept: application/json
```

Le middleware `AuthenticateInstallationToken` accepte aussi, pour compatibilité, `X-MAXIMUS-INSTALLATION-TOKEN` . Il calcule le SHA-256 du secret reçu, cherche une ligne correspondante et refuse si la ligne n'existe pas, si `revoked_at` est défini ou si le statut vaut `REVOKED` .

6.2 Endpoint de configuration

```
GET /api/installation-sync/configuration
```

La route est protégée par `maximus.central` puis par `maximus.installation.token` . L'installation identifiée par le jeton est placée dans les attributs de requête ; le contrôleur ne prend pas le

`companyId` depuis le client.

6.3 Reconstitution des modules

Le central appelle `synchronizedModuleIds()` :

1. il part de `companies.requested_modules` ;
2. il charge les statuts persistés dans `maximus_company_modules` pour l'entreprise ;
3. il conserve les statuts `ACTIF`, `BETA` et `MAINTENANCE` ;
4. il retire les modules dont le statut est `INACTIF` ;
5. il ajoute les modules actifs trouvés dans la table d'accès même s'ils manquent encore dans la demande historique.

Pourquoi cette double source contrôlée ? `requested_modules` reflète l'inscription initiale et peut être ancien. La ligne d'accès persistée est la source de vérité effective. Le contrôleur de modification maintient toutefois aussi `requested_modules` cohérent pour les anciens consommateurs.

6.4 Exemple de réponse anonymisée

```
{
  "configurationVersion": 12,
  "applicationVersion": "<commit-deploy>",
  "syncProtocolVersion": 2,
  "installation": {
    "id": "installation-<uuid>",
    "companyId": "<company-id>",
    "mode": "dedicated"
  },
  "company": {
    "id": "<company-id>",
    "name": "Entreprise exemple",
    "manager": "Responsable",
    "email": "admin@example.invalid",
    "phone": "",
    "country": "",
    "sector": "",
    "loginSlug": "entreprise-exemple",
    "profilePhoto": null,
    "primaryColor": "#F2B705",
    "accentColor": "#F2B705",
    "sidebarColor": "#161D27"
  },
  "modules": {
    "ids": ["commerce", "payroll"],
    "packIds": {"commerce": ["commerce-standard"]},
    "featureIds": {"commerce": ["products", "orders"]},
    "permissions": {"commerce": {"products": ["voir", "modifier"]}}
  },
  "catalog": [],
  "domains": [],
  "erpAccess": {
    "canonicalUrl": "https://erp.example.invalid",
    "allowedHosts": ["erp.example.invalid"]
  }
}
```

Le catalogue est celui publié par le central pour les IDs transmis. Les modules ne sont pas librement importés par le local : `ModuleCatalog::isPublishedModule()` doit les reconnaître après import du catalogue.

6.5 Endpoint heartbeat

```
POST /api/installation-sync/heartbeat
Content-Type: application/json

{"appliedVersion": 12}
```

Après application locale, le local envoie l'ACK. Le central met l'installation à `CONNECTED`, met à jour `last_seen_at`, `last_sync_at` et efface `last_error`. Si l'ACK échoue après une application locale réussie, la configuration n'est pas annulée ; l'erreur est conservée pour le prochain cycle.

7. Traitement local détaillé

7.1 Pipeline exact

```
SyncCentralInstallation::handle()
├─ vérifie InstallationContext::isCompanyOnly()
├─ InstallationSyncService::fetch()
│   ├─ écrit lastAttemptAt + state=syncing
│   ├─ appelle le central en HTTPS
│   ├─ traduit les erreurs HTTP / connexion
│   └─ valide identité, protocole et versions
├─ InstallationSyncService::apply()
│   ├─ revalide le payload
│   ├─ valide erpAccess
│   ├─ ouvre une transaction DB
│   ├─ importe le catalogue publié
│   ├─ normalise packs / features / permissions
│   ├─ met à jour l'entreprise
│   ├─ remplace les accès modules
│   ├─ remplace les domaines e-commerce
│   ├─ commit
│   └─ publie l'état local synced
├─ heartbeat()
└─ code 0 + message de succès
```

7.2 Validation d'identité

Le central peut être joignable sans être autorisé à modifier le local. Le payload doit identifier exactement :

- `company.id` égal à `MAXIMUS_INSTALLATION_COMPANY_ID` ;
- `installation.id` ou `installationId` égal à `MAXIMUS_INSTALLATION_ID` ;
- `installation.companyId` égal à l'entreprise locale ;
- `installation.mode` égal au mode local.

Un écart déclenche `Identité entreprise/installation reçue différente de la configuration locale.` avant toute écriture métier.

7.3 Validation des accès ERP

`erpAccess.allowedHosts` doit être un tableau d'hôtes valides, sans wildcard. L'URL canonique doit être HTTP ou HTTPS, sans identifiant, mot de passe, query string ni fragment, et son hostname doit

apparaître dans `allowedHosts`. Cette validation évite de faire entrer dans le contrôle d'hôte une URL arbitraire issue du réseau.

7.4 Application transactionnelle

L'application est encapsulée dans une transaction. Elle effectue les opérations suivantes :

1. importe le catalogue fourni et s'assure de sa structure locale ;
2. vérifie que chaque module reçu est publié localement ;
3. normalise les packs, fonctionnalités et permissions avec le catalogue ;
4. met à jour ou crée l'entreprise avec statut `ACTIF` ;
5. met à jour les couleurs et le slug de connexion ;
6. ne copie pas une URL de photo centrale et ne télécharge pas de média arbitraire ; les octets d'un logo local sont conservés ;
7. supprime puis recrée les lignes locales de `maximus_company_modules` correspondant à la photo reçue ;
8. remplace les domaines e-commerce locaux de l'entreprise par le snapshot central ;
9. commit la transaction.

Conséquence : le payload est une photo complète pour l'entreprise, pas un patch. Toute modification du code d'application qui ajoute une nouvelle section au snapshot doit préserver l'atomicité et les règles de validation avant écriture.

7.5 Ce qui n'est jamais remplacé par la synchronisation

- les mots de passe administrateur locaux après installation ;
- les sessions locales ;
- les octets de logo téléversés localement ;
- la base locale entière ;
- le logiciel lui-même : une synchronisation de configuration n'est pas une mise à jour de code.

8. Persistance locale et mode hors ligne

`InstallationSyncState` conserve un fichier JSON dans `storage/app/installation-sync/`. Le nom dépend d'un hash de l'entreprise, de l'installation et du mode ; un fichier d'une autre installation n'est donc pas accepté.

Champ	Rôle
<code>lastAttemptAt</code>	Dernière tentative, même échouée.
<code>lastSuccessAt</code>	Dernière configuration appliquée avec succès.
<code>state</code>	État opérationnel courant.
<code>lastError</code>	Message non secret destiné au diagnostic.
<code>versionWarning</code>	Avertissement de divergence de build compatible.
<code>erpAccess</code>	Dernier mapping canonique et liste d'hôtes autorisés.
<code>configurationVersion</code>	Version centrale appliquée.

Les écritures sont protégées par un verrou fichier, réalisées dans un fichier temporaire puis publiées par renommage atomique. Les permissions du fichier temporaire sont fixées à `0600`.

En cas d'échec réseau, le service enregistre `unreachable` et ne supprime ni l'entreprise ni les modules ni le mapping ERP déjà appliqués. Le serveur local continue donc à servir sa dernière configuration valide.

9. Scheduler Windows

9.1 Fonctionnement

`routes/console.php` définit la tâche Laravel toutes les cinq minutes et la conditionne au mode entreprise. `scripts/register-maximus-sync.ps1` crée la tâche Windows `MAXIMUS_central_scheduler` :

- déclenchement par répétition toutes les cinq minutes ;
- tâche marquée cachée ;
- PowerShell lancé avec `-WindowStyle Hidden` ;
- processus PHP lancé avec `-WindowStyle Hidden` ;
- limite d'exécution de deux minutes ;
- verrou Laravel `withoutOverlapping(60)` pour éviter deux synchronisations simultanées.

Une tâche déjà enregistrée conserve son ancienne fréquence. Après une mise à jour du script, il faut donc la réenregistrer. Une ancienne tâche réglée à une minute peut expliquer des fenêtres fréquentes ou une charge inutile.

9.2 Résolution DNS Windows

Le client PHP cURL peut avoir un comportement différent de `curl.exe` sous Windows. Dans le code actuel, les appels centraux préfèrent IPv4 et réessaient les exceptions de connexion transitoires. Les réponses HTTP `401`, `403` et `5xx` ne sont pas masquées : elles conservent leur signification métier.

10. Sécurité

10.1 Jeton

- génération aléatoire côté central ;
- stockage central par hash SHA-256, jamais en clair ;
- transport par HTTPS et en-tête Bearer ;
- rotation par réémission ;
- révocation centrale immédiate ;
- absence dans les messages d'erreur et dans l'état local public.

10.2 Isolation tenant

Le `companyId` local est lu depuis la configuration serveur. Le payload central n'a pas le pouvoir de changer le périmètre : il doit correspondre à cette valeur. L'installation ne choisit pas une autre entreprise en modifiant une URL ou une query string.

10.3 Réduction de surface

- les routes de sync sont limitées au mode central et au middleware de jeton ;

- les installations révoquées ne sont plus acceptées ;
- les hôtes ERP sont validés et les wildcards sont refusés ;
- les médias centraux ne sont pas téléchargés automatiquement ;
- le code local ne crée jamais d'administrateur global MAXIMUS.

11. États et erreurs

Symptôme / HTTP	État	Interprétation	Action
DNS, timeout, cURL error 28	unreachable	Le transport n'a pas atteint le central.	Tester DNS/HTTPS, vérifier PHP cURL, laisser le retry et conserver la dernière config.
401 sans code de révocation	authentication_rejected	Jeton absent, invalide ou réémis.	Ne pas modifier la base locale ; réémettre un bootstrap.
401 + INSTALLATION_REVOKED	revoked	L'installation a été explicitement révoquée.	Stopper l'exploitation dédiée et traiter l'enrôlement depuis MAXIMUS.
409 entreprise inactive	rejected	L'entreprise centrale n'est plus active.	Réactiver ou corriger l'état métier côté central.
Protocole incompatible	rejected	Le local et le central ne parlent pas le même contrat.	Déployer une version compatible, sans forcer la base.
Identité différente	rejected	Le bootstrap, le .env ou le central ne désignent pas la même installation.	Comparer les IDs non secrets et régénérer le bootstrap si nécessaire.
Module non publié localement	rejected	Le catalogue reçu ne peut pas matérialiser un module.	Publier un catalogue compatible et refaire la synchronisation.
Erreur de transaction	rejected	La nouvelle configuration n'est pas appliquée intégralement.	La transaction est annulée ; diagnostiquer sans supprimer la dernière configuration.

12. Versionnement et compatibilité

Deux notions sont distinctes :

Version	Usage
syncProtocolVersion	Version du contrat de synchronisation. Elle vaut actuellement 2. Un écart est bloquant.
applicationVersion	Identifiant du code déployé au central, généralement le commit Git.
MAXIMUS_BUILD_VERSION	Identifiant du package logiciel local.
configurationVersion	Compteur de configuration d'installation côté central. Il est renvoyé et mémorisé localement.

Point d'attention d'architecture : dans le code actuel, `configurationVersion` est incrémenté lors de l'émission d'une installation et lors des changements d'adresses ERP. Les changements d'accès modules sont récupérés par le polling complet même s'ils ne font pas nécessairement progresser ce compteur. Il ne faut donc pas utiliser ce numéro comme unique condition pour décider de ne pas appeler le central.

13. Procédures d'exploitation

13.1 Vérifier le central

```
curl.exe -i --connect-timeout 10 --max-time 30 `
https://maximus-erp.onrender.com/api/healthz
```

Le résultat attendu est HTTP 200 avec `ok=true`, `database=true`, `status=OPERATIONAL` et un `applicationVersion` non vide.

13.2 Forcer une synchronisation locale

```
cd C:\Chemin\vers\maximus-local-propre\artifacts\api-server\laravel
php artisan config:clear
php artisan maximus:sync-central-installation
```

Le succès affiche `Configuration centrale synchronisée pour ...`. La commande ne fait ni `git pull`, ni build, ni migration, ni mise à jour du logiciel.

13.3 Réenregistrer le scheduler Windows

```
cd C:\Chemin\vers\maximus-local-propre
powershell -NoProfile -ExecutionPolicy Bypass `
-File .\scripts\register-maximus-sync.ps1 `
-WorkspaceDir (Get-Location).Path `
-IntervalMinutes 5
```

Cette commande remplace la tâche portant le même nom. Elle est nécessaire après un changement de fréquence ou de comportement caché.

13.4 Désactiver temporairement la tâche

```
Disable-ScheduledTask -TaskName "MAXIMUS central scheduler"
```

Cette action arrête la synchronisation automatique mais ne supprime pas la configuration locale. Pour remettre la tâche, réexécuter l'enregistrement ci-dessus.

13.5 Vérifier l'absence de fuite de secret

- ne pas afficher `MAXIMUS_INSTALLATION_TOKEN` ;
- ne pas envoyer le bootstrap dans un ticket ou un dépôt ;
- ne pas utiliser `git diff` sur un fichier `.env` dans un canal partagé ;
- ne publier que l'URL, l'ID d'entreprise, l'ID d'installation et les codes HTTP nécessaires au diagnostic.

14. Cartographie du code

Fichier	Responsabilité
app/Http/Controllers/Api/InstallationController.php	Émission, renouvellement, révocation et construction du payload central.
app/Http/Controllers/Api/InstallationAccessController.php	Adresses ERP, vérification DNS/HTTPS, adresse primaire et compteur de configuration.
app/Http/Middleware/AuthenticateInstallationToken.php	Vérification Bearer/X-Header, hash et révocation.
app/Services/InstallationSyncService.php	Client HTTP, validation, transaction d'application, état local et heartbeat.
app/Support/InstallationContext.php	Mode, entreprise configurée, installation configurée, hôtes de confiance et entrée utilisateur.
app/Support/InstallationSyncState.php	Snapshot JSON local, verrou fichier, état opérationnel sans secret.
app/Support/ApplicationIdentity.php	Version de build, version attendue et protocole v2.
app/Console/Commands/InstallCompanyInstance.php	Bootstrap initial, création/conserver de l'administrateur entreprise.
app/Console/Commands/SyncCentralInstallation.php	Commande périodique : fetch, apply, heartbeat.
routes/api.php	Routes d'administration et routes installation-sync.
routes/console.php	Planification Laravel toutes les cinq minutes.
scripts/install-maximus-instance.sh	Installation de l'archive dédiée et écriture sécurisée du .env.
scripts/package-maximus-instance.sh	Création d'une archive seulement si le commit local correspond au central.
scripts/register-maximus-sync.ps1	Tâche Windows cachée, cadence de cinq minutes.
database/migrations/2026_09_16_000048_create_maximus_installations_table.php	Identité, hash du jeton, statut, version et observabilité d'installation.
database/migrations/2026_09_16_000049_create_installation_addresses.php	Plusieurs adresses ERP par installation et adresses

15. Invariants à préserver lors d'une évolution

1. Ne jamais prendre le tenant depuis une valeur envoyée par le navigateur ou un paramètre de synchronisation.
2. Ne jamais stocker le jeton central en clair côté central.
3. Ne jamais appliquer une partie du payload avant d'avoir validé l'identité et le protocole.
4. Ne jamais remplacer la dernière configuration locale par une réponse réseau partielle ou invalide.
5. Ne jamais confondre synchronisation de configuration et mise à jour du logiciel.
6. Ne jamais utiliser uniquement `requested_modules` pour calculer les modules effectivement autorisés.
7. Ne jamais supposer que `configurationVersion` est un curseur complet de toutes les modifications métier.
8. Ne jamais afficher le jeton dans une exception, un log, un PDF, un ticket ou un commit.
9. Conserver la compatibilité de `syncProtocolVersion` ou faire évoluer central et package local ensemble.
10. Tester les modes hors ligne, token révoqué, identité incorrecte, catalogue incomplet, timeout DNS et double exécution du scheduler.

Conclusion opérationnelle : une installation dédiée est une instance autonome pour l'exécution quotidienne, mais son autorité de configuration reste centrale. Le mécanisme est un pull périodique authentifié, borné par tenant, validé par version et appliqué atomiquement. La panne réseau conserve la dernière vérité locale connue ; la révocation et les incohérences d'identité, elles, doivent rester visibles et bloquantes.